

GreenBite

Plan de Branching y Estrategia de Control de Versiones

1. Estrategia de ramas adoptada

El equipo de GreenBite adopto una estrategia de ramas basada en Feature Branching. Cada componente del sistema se desarrollo en una rama independiente y se integro a la rama principal (main) mediante merge una vez que el componente estuvo estable y probado.

2. Estructura de ramas

Rama	Proposito	Estado
main	Codigo de produccion, entrega final	Activa
feature/frontend	Desarrollo de la interfaz React de GreenBite	Mergeada
feature/backend-para-frontend	Capa de agregacion BFF	Mergeada
feature/producto-service	Microservicio del catalogo de cajas	Mergeada
feature/compra-service	Microservicio de suscripciones	Mergeada

3. Flujo de trabajo

El flujo seguido para cada componente fue el siguiente:

Paso 1: Crear la rama

```
git checkout -b feature/nombre-componente
```

Paso 2: Desarrollar el componente

Implementar el codigo, pruebas y README

Paso 3: Hacer commit en la rama

```
git add . && git commit -m "feat: descripcion"
```

Paso 4: Volver a main

```
git checkout main
```

Paso 5: Mergear la rama

```
git merge feature/nombre-componente
```

Paso 6: Subir a GitHub

```
git push origin main
```

4. Commits realizados por rama

Cada rama tuvo al menos un commit significativo antes de ser integrada a main. Los mensajes de commit siguen la convencion feat: para nuevas funcionalidades y docs: para documentacion.

Rama	Mensaje del commit
feature/frontend	feat: desarrollo componente frontend GreenBite
feature/backend-para-frontend	feat: desarrollo capa Backend Para Frontend
feature/producto-service	feat: desarrollo microservicio de productos
feature/compra-service	feat: desarrollo microservicio de compras

5. Justificacion de la estrategia

Se eligio Feature Branching porque el proyecto GreenBite tiene componentes claramente delimitados (frontend, BFF, productos, compras) que pueden desarrollarse en paralelo sin generar conflictos entre ellos. Cada integrante del equipo puede trabajar en su componente sin afectar el trabajo de los demas.

Esta estrategia tambien facilita la revision del codigo antes de mergear, lo que reduce los errores en la rama principal y permite identificar facilmente desde que rama proviene cada cambio en el historial de Git.

6. Verificacion en GitHub

La evidencia de las ramas y merges puede verificarse en:

<https://github.com/Truquitos212/evaluacion>

En la seccion 'Branches' del repositorio se visualizan las 4 ramas feature y en el historial de commits de main se pueden ver los 4 merges realizados.